

< / **steam ducks** >

SCA — Sistema de Controle e Acompanhamento

Documento Técnico de Entrega

Infraestrutura · DevOps · Monitoramento · Capacity Planning · Custos

Versão 1.0 · Junho de 2026 · Confidencial

1. Visão geral do projeto

Este documento formaliza a entrega técnica do sistema SCA (Sistema de Controle e Acompanhamento), desenvolvido pela Steam Ducks. Seu objetivo é registrar as decisões de arquitetura, a infraestrutura provisionada, o pipeline de CI/CD implementado, a estratégia de monitoramento e as projeções de capacidade e custo para os próximos 17 meses de operação.

O documento destina-se ao time de desenvolvimento e DevOps responsável por operar, manter e evoluir o sistema após a entrega. Recomenda-se sua consulta em três momentos distintos: no recebimento do sistema (leitura completa), antes de cada decisão de escala de infraestrutura (seções 4 e 5) e diante de incidentes em produção (seção 7).

Repositório e acessos

Item	Valor
Repositório	github.com/Steam-Ducks/sca-server
Branch prod	main
Branch dev	develop
VPS (prod)	143.198.2.189 — Digital Ocean · NYC3 · Ubuntu 24.04 LTS
Grafana	http://143.198.2.189:3000 — credenciais entregues separadamente
Staging	http://143.198.2.189:8001
Prod	http://143.198.2.189:8000

2. Arquitetura da aplicação

A aplicação roda inteiramente no droplet entregue. O PostgreSQL é o único componente externo — provisionado como serviço gerenciado na Digital Ocean e acessado via TCP. Essa separação garante que falhas no droplet de aplicação não comprometam os dados, e que o banco possa ser escalado independentemente da aplicação.

Componentes e responsabilidades

Componente	Tecnologia	Papel	Porta / Acesso
Backend	Django + Gunicorn	API HTTP — processa todas as regras de negócio	8000 (interno ao droplet)

Componente	Tecnologia	Papel	Porta / Acesso
Cache	Redis 7	Cache de sessões e respostas; reduz carga no banco	6379 (interno)
Banco	PostgreSQL 18 (DO)	Persistência de todos os dados da aplicação	TCP/5432 (externo)
Proxy reverso	nginx	Termina conexões HTTP, serve estáticos, roteia /api/*	80 / 443 (público)
Métricas	Prometheus	Coleta métricas da app e do servidor a cada 15 segundos	9090 (interno)
Dashboards	Grafana Cloud	Visualização de métricas, alertas e capacity planning	3000 (público)
Alertas	Alertmanager	Roteamento e disparo de alertas preditivos	9093 (interno)

Decisões de arquitetura

As principais decisões tomadas durante o projeto, com suas justificativas:

Por que Django + Gunicorn e não Node.js ou FastAPI?

O time já possuía domínio sobre Django. A escolha prioriza velocidade de entrega e manutenibilidade. O Gunicorn com workers síncronos é suficiente para o perfil de carga previsto até o mês 11. Se a aplicação evoluir para operações assíncronas intensas, considerar migração de workers para Uvicorn/Asgi.

Por que Redis como cache e não Memcached?

O Redis suporta estruturas de dados mais ricas (listas, sets, hashes) e persistência opcional, o que permite uso futuro como fila de tarefas com Celery sem adicionar nova dependência de infraestrutura.

Por que PostgreSQL gerenciado e não instalado no droplet?

O serviço gerenciado da Digital Ocean inclui backups automáticos diários, failover automático e patches de segurança sem intervenção manual. O custo adicional de \$15,15/mês é justificado pela eliminação do risco de perda de dados por falha operacional.

Por que Prometheus + Grafana e não Datadog ou New Relic?

O stack Prometheus + Grafana é open source, sem custo de licença por host e com controle total sobre retenção de dados. O Grafana Cloud no plano Free é suficiente para os primeiros meses. Datadog e New Relic cobram por host e por volume de logs — o custo escalaria junto com o crescimento da aplicação.

3. Pipeline de CI/CD

O pipeline foi construído inteiramente no GitHub Actions e implementa o princípio de shift-left: erros são detectados o mais cedo possível no fluxo, antes de chegarem ao ambiente de produção. Nenhuma alteração vai para produção sem passar por todas as etapas abaixo.

Fluxo completo

Etapa	Job	O que faz	Bloqueia se...
1	lint	Verifica formatação e estilo do código Python (Ruff)	Código fora do padrão
2	unit-tests	Testa funções isoladas sem dependências externas	Qualquer teste falhar
3	integration-tests	Testa endpoints com banco e Redis reais (settings_test)	Qualquer teste falhar
4	smoke-test	Verifica 20 endpoints críticos retornando status 2xx após o build	Qualquer endpoint falhando
5	load-test	k6: 5 VUs por 2 min em endpoints reais — verifica p95 e error rate	p95 > 2.000ms ou error rate > 1%
6	auto-merge	Faz merge automático em develop se todas as etapas anteriores passaram	Qualquer etapa anterior falhou

Fluxo de deploy

O deploy segue um fluxo de promoção controlada, onde cada ambiente precisa ser validado antes de avançar para o próximo:

Etapa	Trigger	Ambiente	Condição para avançar
1 — CI	Pull Request → develop	Runner GitHub Actions	Todos os 5 jobs passarem
2 — Staging	Merge em develop	VPS porta 8001	Smoke test pós-deploy passar
3 — Aprovação	Automático	GitHub (pausa aqui)	Aprovação manual de reviewer obrigatório
4 — Prod	Após aprovação	VPS porta 8000	—

Como aprovar: Acesse github.com/Steam-Ducks/sca-server → Actions → Deploy Staging → Review deployments → selecione 'production' → Approve.

Hotfix de emergência: Acesse Actions → staging.yml → Run workflow → selecione a branch de correção → Run. O fluxo normal de aprovação ainda se aplica.

⚠ Risco: Nunca faça deploy direto via SSH em produção. Isso bypassa os testes e o histórico de versões no GitHub, tornando impossível rastrear o que foi para produção e quando.

4. Cenário de uso e capacity planning

O SCA é uma aplicação de uso interno, com acesso concentrado no horário comercial. Esse perfil é favorável: a carga é previsível, sem picos noturnos ou de fim de semana, o que facilita tanto o dimensionamento inicial quanto o planejamento de manutenções.

Projeção de crescimento

Marco	Usuários simultâneos	Contexto
Mês 1	~200	Início de operação — time interno completo usando o sistema
Mês 11	~4.000	Crescimento acelerado — base de usuários expandida
Mês 17	~20.000	Alta escala — ponto crítico que exige decisão de arquitetura

Metodologia: A extrapolação parte do comportamento medido nos load tests com 5 VUs e aplica escala linear ajustada pelo padrão de uso diurno. Os valores são estimativas conservadoras — o comportamento real pode variar conforme o perfil de uso.

Previsão de recursos por marco

Marco	Usuários	CPU	RAM	p95 est.	Disco	Veredito
Mês 1	200	~15%	~2,0 GB	~200 ms	~21 GB	OK
Mês 6	~2.000	~45%	~2,8 GB	~600 ms	~35 GB	OK
Mês 11	~4.000	~70%	~3,4 GB	~1.200 ms	~52 GB	Atenção
Mês 14	~10.000	~90%	~3,7 GB	~2.500 ms	~70 GB	Crítico
Mês 17	~20.000	Saturada	Saturada	> 5.000 ms	~90 GB	Inviável

⚠ Risco: Os valores marcados como Crítico e Inviável assumem ausência de intervenção. Com o upgrade planejado executado antes do mês 11, todos os indicadores voltam à faixa verde.

5. Resultados dos testes de carga

Os testes de carga foram executados com k6 diretamente no pipeline de CI, garantindo que cada Pull Request seja validado sob carga antes de ser mergeado. Os resultados abaixo referem-se à execução contra o ambiente de CI com banco e Redis reais.

Configuração do teste

Parâmetro	Valor e justificativa
Ferramenta	k6 — binário standalone, sem dependências Python, integração nativa com GitHub Actions
VUs simultâneos	5 VUs — representa carga leve de CI; testes de carga completos ficam em cenários de staging
Duração	2 minutos (30s ramp-up · 1min plateau · 30s ramp-down)
Threshold p95	< 2.000ms — calibrado para runners compartilhados do GitHub (não reflete prod real)
Threshold error	< 1% — qualquer degradação de disponibilidade bloqueia o merge
Autenticação	AllowAny via settings_test — auth JWT completa testada no post-deploy-smoke.yml

Resultados obtidos

Métrica	Valor medido	Threshold	Situação
Throughput	22,6 RPS	—	—
Latência média	59 ms	—	—
Latência p90	83 ms	—	—
Latência p95	93 ms	< 2.000 ms	✓ OK
Latência máxima	273 ms	—	—
Error rate	0,00%	< 1%	✓ OK
Checks passados	100%	100%	✓ OK

Métrica	Valor medido	Threshold	Situação
Total de requests	2.728	—	—

Interpretação: O p95 de 93ms com 5 VUs significa que 95% das requisições responderam em menos de 93ms. A margem para o threshold de 2.000ms é de 21x, indicando que a aplicação tem folga confortável para crescimento antes que o CI comece a sinalizar degradação.

6. Gargalo previsto e recomendação de escala

Com base nos testes de carga e na projeção de crescimento, o gargalo esperado é exclusivamente de CPU. O Unicorn processa requisições de forma síncrona e ocupa um núcleo por worker. Com 2 vCPUs, a saturação ocorre ao redor de 4.000 usuários simultâneos — previsto para o mês 11.

RAM e disco não são gargalos para a janela de 17 meses: a RAM tem 2,0 GB livres e o disco tem 96 GB disponíveis com crescimento estimado de ~4 GB/mês de dados.

Simulação de carga por configuração

Cenário	vCPUs	p95 est.	Throughput	Veredito
200 usuários — mês 1	2	~200 ms	~22 RPS	OK — adequado até mês 11
4.000 usuários — mês 11	2	~1.200 ms	~22 RPS	No limite — monitorar semanalmente
20.000 usuários — mês 17	2	> 5.000 ms	Saturado	Inviável — upgrade obrigatório antes
20.000 usuários com upgrade	8	~400 ms	~80 RPS	OK — configuração recomendada pós-mês 11

Plano de ação por gatilho

Quando	Gatilho observado no Grafana	Ação recomendada
Mês 9–10	CPU média diária > 60% por 3 dias seguidos	Abrir issue de planejamento de upgrade — avaliar janela de manutenção
Mês 11	CPU > 70% contínuo ou p95 > 1.000ms	Executar resize do droplet para General Purpose 4 vCPU (\$63/mês)
Mês 14+	p95 > 2.000ms ou error rate > 1% por 15min	Avaliar load balancer + múltiplas instâncias ou migração para k8s

Quando	Gatilho observado no Grafana	Ação recomendada
Qualquer	Disco > 70% usado	Limpar imagens Docker antigas: <code>docker system prune -af --volumes</code>

Resize do droplet: O resize na Digital Ocean não causa perda de dados. O tempo de indisponibilidade é de aproximadamente 2 minutos. Recomenda-se executar fora do horário comercial.

⚠ Risco: Não aguardar o sistema saturar para agir. O degradação de performance percebida pelos usuários começa quando a CPU ultrapassa 70% — antes de qualquer alerta crítico ser disparado.

7. Monitoramento e runbook de alertas

A stack de monitoramento foi configurada para ser completamente autônoma: coleta métricas automaticamente, dispara alertas preditivos antes que problemas se tornem incidentes e persiste dados por 90 dias no Prometheus local com `remote_write` para o Grafana Cloud.

Stack de monitoramento

Componente	Tecnologia	O que coleta / faz	Onde ver
Métricas de app	django-prometheus	Requests, latência, status codes, queries ao banco, cache hits	Grafana → SCA Aplicação
Métricas de infra	node-exporter	CPU, RAM, disco, rede, uptime do servidor	Grafana → SCA Infraestrutura
Quality Gate	Grafana dashboard	Comparação staging vs prod antes de cada aprovação de deploy	Grafana → Quality Gate
Capacity Planning	<code>predict_linear</code> (PromQL)	Projeção de disco, RAM e CPU para 30 e 90 dias	Grafana → Capacity Planning
Alertas preditivos	Alertmanager	7 regras — disparam antes do problema virar incidente	Dashboard Grafana

Dashboards disponíveis

Acesse <http://143.198.2.189:3000> → Dashboards → SCA para encontrar os quatro painéis abaixo:

Dashboard	Quando usar
SCA — Aplicação	Visão em tempo real da saúde da API: requests/s, latência p95, error rate, cache hit rate. Consultar após qualquer incidente ou deploy.
SCA — Infraestrutura	CPU, RAM e disco do servidor. Consultar quando a latência estiver alta para identificar se o gargalo é de recurso ou de código.
SCA — Quality Gate	Comparação lado a lado de staging vs prod. Abrir ANTES de aprovar qualquer deploy. Se qualquer métrica de staging estiver pior que prod, não aprovar.
SCA — Capacity Planning	Projeções de disco, RAM e CPU para os próximos 30 e 90 dias. Consultar mensalmente e sempre que um alerta preditivo for disparado.

Runbook de alertas

Os alertas abaixo são disparados pelo Alertmanager e aparecem no dashboard do Grafana. Para cada alerta, siga o procedimento descrito:

● AppForaDoAr — CRÍTICO

Significa: o Prometheus não consegue mais coletar métricas do container de produção. A aplicação está inacessível para os usuários.

Procedimento:

1	ssh root@143.198.2.189
2	docker compose -f /opt/sca/docker-compose.prod.yml ps
3	docker compose -f /opt/sca/docker-compose.prod.yml logs web --tail=100
4	Se container parado: docker compose -f /opt/sca/docker-compose.prod.yml restart web
5	Se não resolver: verificar logs do banco — docker compose logs db --tail=50

● ErrorRateAlto — CRÍTICO

Significa: mais de 1% das requisições estão retornando erro 5xx por mais de 5 minutos. Usuários estão recebendo erros.

1	Abrir Grafana → SCA Aplicação → painel Respostas por status code para identificar qual endpoint está falhando
2	docker compose -f /opt/sca/docker-compose.prod.yml logs web --tail=200 -f
3	Verificar se o erro começou após um deploy recente — consultar GitHub Actions → histórico
4	Se sim: fazer rollback via GitHub Actions → staging.yml → Run workflow na tag anterior

● LatenciaAltaP95 — ATENÇÃO

Significa: 5% das requisições estão demorando mais de 2.000ms por mais de 5 minutos. A aplicação está lenta mas disponível.

1	Abrir Grafana → SCA Infraestrutura — verificar se CPU está acima de 80%
2	Se CPU alta: <code>docker stats --no-stream</code> para ver qual processo consome mais
3	Abrir Grafana → SCA Aplicação → Queries ao banco — verificar se há pico de queries
4	Se queries altas: investigar N+1 queries no código — adicionar <code>select_related/prefetch_related</code>

● DiscoEnchendoEm30Dias — ATENÇÃO

Significa: no ritmo atual de crescimento, o disco vai atingir 85% de uso em menos de 30 dias. Não é urgente, mas requer ação planejada.

1	<code>ssh root@143.198.2.189 && du -sh /opt/sca/* sort -rh head -20</code>
2	Limpar imagens Docker antigas: <code>docker system prune -af --volumes</code> (libera espaço sem afetar containers rodando)
3	Verificar logs grandes: <code>find /opt/sca -name '*.log' -size +50M</code>
4	Se não resolver: expandir volume no painel Digital Ocean → Droplets → Resize (sem downtime para disco)

● RAMAltaContínua / CPUAltaContínua — ATENÇÃO

Significa: uso de recurso acima do threshold por tempo prolongado. Pode indicar memory leak, processo zumbi ou necessidade de escala.

1	<code>docker stats --no-stream</code> — identificar qual container consome mais
2	Se for o container web: verificar se há requests longas presas — <code>docker logs web --tail=200</code>
3	Reiniciar workers se necessário: <code>docker compose restart web</code> (downtime de ~5 segundos)
4	Se persistir por mais de 2 dias: consultar seção 6 deste documento — pode ser hora de escalar

8. SLOs — Objetivos de Nível de Serviço

Os SLOs abaixo definem os alvos operacionais acordados para o sistema em produção. Violações pontuais são aceitáveis; violações recorrentes de um mesmo SLO indicam necessidade de investigação da causa raiz.

Métrica	SLO	Alerta dispara em	Severidade	Consequência se violado
Uptime	≥ 99,5%/mês	App fora por > 1 min	Crítico	Usuários sem acesso ao sistema
Latência p95	< 2.000 ms	5 min acima do limite	Atenção	Experiência degradada — sistema lento
Error rate	< 1%	5 min acima de 1%	Crítico	Usuários recebendo erros nas operações
CPU	< 80% contínuo	10 min acima de 80%	Atenção	Risco de saturação — latência crescente
RAM	< 85% contínuo	15 min acima de 85%	Atenção	Risco de OOM — container pode ser kilado
Disco	> 15% livre	Previsão < 30 dias p/encher	Atenção	Risco de falha total se disco encher

Uptime 99,5%: Corresponde a no máximo 3h 39min de indisponibilidade por mês. Para alcançar 99,9% seria necessário load balancer com múltiplas instâncias — fora do escopo atual.

9. Custos de infraestrutura

Todos os custos abaixo referem-se a serviços da Digital Ocean e Grafana Cloud, cobrados em dólar americano (USD). Os valores são públicos e verificáveis nos painéis de cada serviço.

9.1 Custo atual — baseline

Serviço	Configuração atual	Custo mensal (USD)	Custo anual (USD)
Droplet de aplicação	2 vCPU · 3,8 GB RAM · 116 GB SSD · NYC3	\$ 32,00	\$ 384,00
PostgreSQL gerenciado	1 vCPU · 1 GB RAM · 10 GB · NYC3 · PG 18	\$ 15,15	\$ 181,80
Grafana Cloud	Plano Free — 10k series · 14 dias retenção	\$ 0,00	\$ 0,00
TOTAL ATUAL	—	\$ 47,15	\$ 565,80

9.2 Projeção de custos por marco

Marco	Usuários	Configuração prevista	Custo mensal	Ação necessária
Mês 1	200	Droplet Basic \$32 + DB \$15,15 + Grafana Free	\$ 47,15	Nenhuma
Mês 6	~2.000	Droplet Basic \$32 + DB \$15,15 + Grafana Free	\$ 47,15	Monitorar CPU no Grafana
Mês 9–10	~3.000	Droplet Basic \$32 + DB \$15,15 + Grafana Free	\$ 47,15	Planejar e orçar o upgrade
Mês 11	~4.000	Droplet GP 4vCPU \$63 + DB \$15,15 + Grafana Free	\$ 78,15	Executar resize do droplet
Mês 14	~10.000	Droplet GP 8vCPU \$126 + DB 2vCPU \$30 + Grafana Free	\$ 156,00	Upgrade do banco + droplet
Mês 17	~20.000	Droplet 8vCPU \$126 + DB \$30 + Grafana Pro \$29	\$ 185,00	Reavaliar arquitetura completa

9.3 Detalhamento do upgrade crítico — mês 11

O upgrade do mês 11 é a única intervenção de custo obrigatória na janela de 17 meses. Abaixo o comparativo antes e depois:

Item	Configuração atual	Após upgrade	Impacto no custo
Droplet	Basic 2 vCPU — \$32,00/mês	General Purpose 4 vCPU — \$63,00/mês	+ \$31,00/mês
PostgreSQL	1 vCPU 1 GB — \$15,15/mês	Sem alteração — \$15,15/mês	\$ 0,00
Grafana	Free — \$0,00/mês	Sem alteração — \$0,00/mês	\$ 0,00
TOTAL	\$ 47,15/mês	\$ 78,15/mês	+ \$31,00/mês

9.4 Resumo de investimento total — 17 meses

Período	Usuários	Custo/mês	Custo do período	Infraestrutura
Mês 1–10	200–3.000	\$ 47,15	\$ 471,50	Sem alteração
Mês 11–16	4.000–15.000	\$ 78,15	\$ 469,00	Droplet 4 vCPU

Período	Usuários	Custo/mês	Custo do período	Infraestrutura
Mês 17	~20.000	~\$185,00	~\$ 185,00	Alta escala
TOTAL 17 MESES	—	—	~ \$1.125,50	Estimativa conservadora

Observação: Valores em USD. Não inclui transferência de dados acima da franquia inclusa (500 GB/mês no droplet atual). Preços da Digital Ocean verificados em junho de 2026.

10. Rotina de operação recomendada

A rotina abaixo foi definida para manter a saúde do sistema com o mínimo de esforço operacional. Todas as verificações são feitas no Grafana — sem necessidade de acesso ao servidor para o acompanhamento rotineiro.

Frequência	Tempo estimado	O que verificar	Onde
A cada deploy	5 minutos	Abrir Quality Gate — comparar staging vs prod. Aprovar só se todas as métricas do staging estiverem iguais ou melhores que prod.	Grafana → Quality Gate
Semanal	15 minutos	Verificar tendência de latência e error rate da semana. Qualquer métrica com tendência crescente por 3+ dias merece investigação.	Grafana → Aplicação
Mensal	30 minutos	Checar painéis 'Dias até 85% de disco' e 'Dias até 90% de RAM'. Se qualquer um estiver abaixo de 60 dias, abrir issue de planejamento.	Grafana → Capacity Planning
Mês 9–10	1 hora	Avaliação de capacidade: CPU média diária. Se > 60% por semana completa, iniciar processo de upgrade conforme seção 6.	Grafana → Infraestrutura

11. Responsabilidades pós-entrega

Item	Responsável	Observação
Servidor — droplet Digital Ocean	Cliente	Acesso ao painel cloud.digitalocean.com — renovação de plano e billing
PostgreSQL gerenciado	Cliente	Configurar política de backup no painel DO — mínimo 7 dias de retenção recomendado
Deploy de atualizações	Equipe técnica	Via GitHub Actions — aprovação obrigatória antes de ir para produção
Monitoramento e dashboards	Grafana Cloud	Automático — nenhuma intervenção necessária para coleta de métricas
Resposta a alertas	Equipe técnica	Seguir runbook da seção 7 deste documento
Upgrade de infraestrutura	Cliente + equipe	Decisão conjunta — gatilhos definidos na seção 6
Backup do banco	Cliente	Digital Ocean faz backup automático diário — configurar retenção mínima de 7 dias
Renovação de secrets e certificados	Equipe técnica	Revisar anualmente — SECRET_KEY e certificados SSL